```
============================================================
 checked : 500
 errors  : 0
 coverage: 100.0%
============================================================
 PASS
```

Bug Log

1. RTL: PE stalled for a cycle after weight switch. I thought this needed to be done sequentially and the stall was necessary to capture the value. I fixed this by using combinational weight promotion so that the result is calculated using the background weight on the same cycle that pe_switch_in goes high.
2. RTL: Weight promotion did not occur (switch/accept signals ignored) while PE was disabled. I assumed the weights stored should freeze while the PE was disabled. I removed the enabled condition for accepting/promoting weights.
3. RTL: PE holds its outputs when disabled. I assumed the PE should hold its outputs when disabled, but they are supposed to be driven to 0 when disabled. I fixed this by checking if the PE is disabled and driving the outputs to 0 if true.
4. Testbench: mon2scb and observed/expected were of type bit, but DUT output is logic. Changed mon2scb and observed/expected to type logic.

Vendor PE Bug Report

Results on Vendor PE

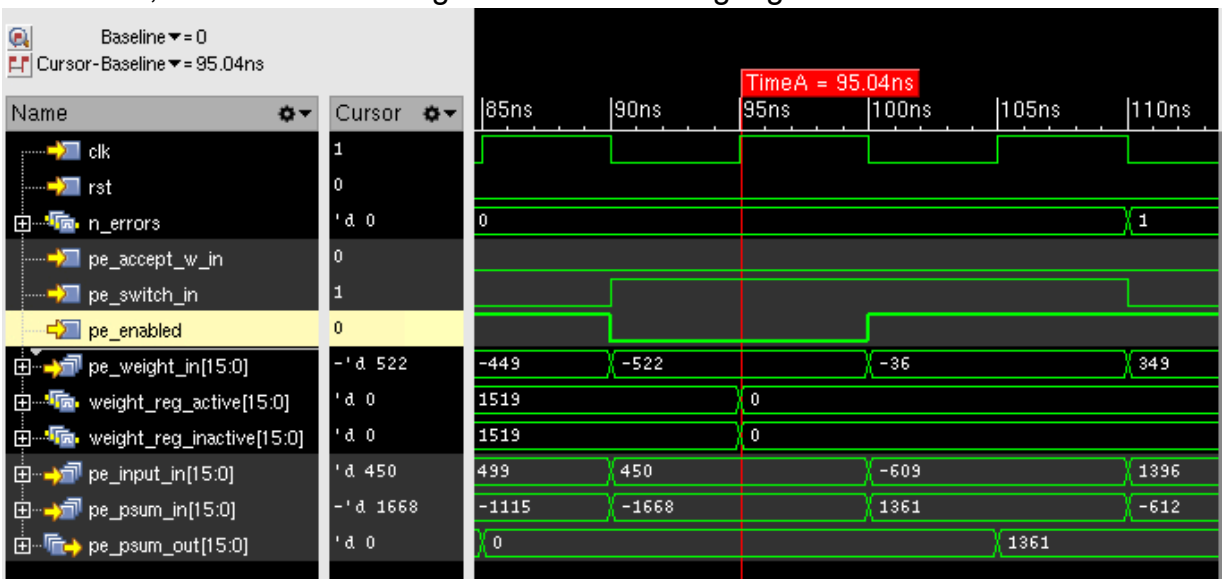
```
================
 checked : 500
 errors  : 297
```

Vendor PE Bugs

Weights go to 0 when pe_enabled is low

```
MISMATCH #8: w=   -36 a=  -609 p=  1361 | acc=0 sw=1 vld=1 en=1 expected=-2253 observed=1361
```

In the waveform, when pe_enabled is low, at the next clock edge (95 ns) when the PE is disabled, both the active weight and inactive weight go to 0.

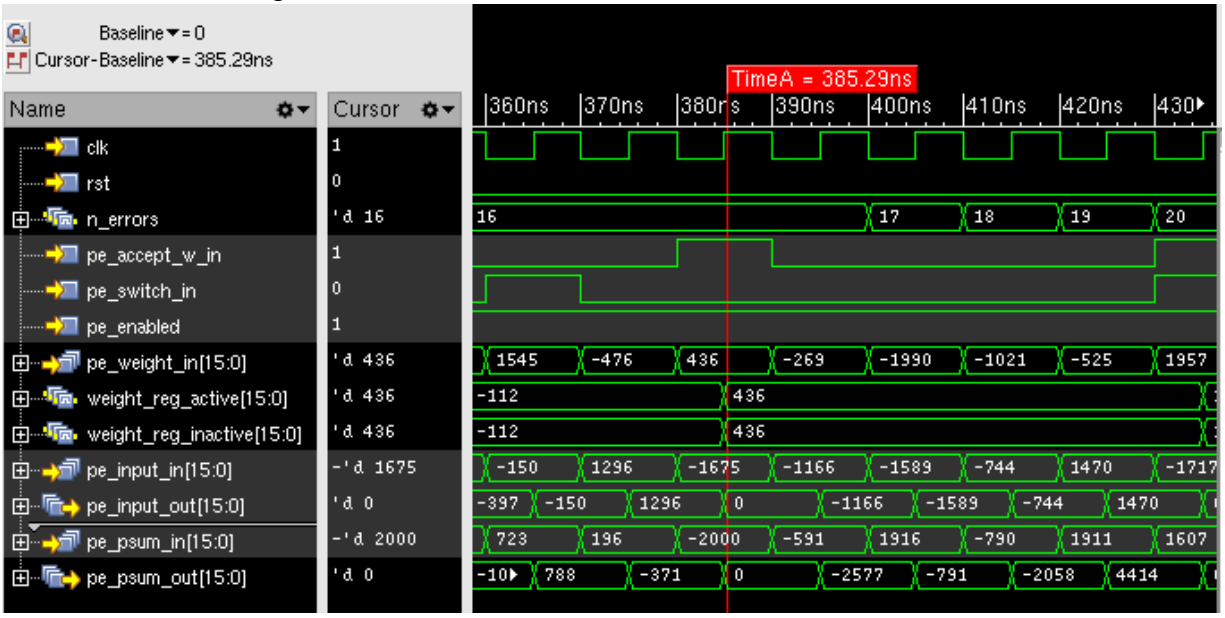


Spec this violates: Alan: “When pe_enabled is 0 all five outputs should go to 0 that cycle, and only weight_bg and the active weight keep their values.“
Fix: Do not clear weight_reg_inactive when pe_enabled is low

Accepting a weight changes both the active and inactive weights.

```
MISMATCH #37: w=  -269 a= -1166 p=  -591 | acc=0 sw=0 vld=1 en=1 expected=-81 observed=-2577
```

At the cursor, a weight should be loaded into inactive, but active is also loaded.

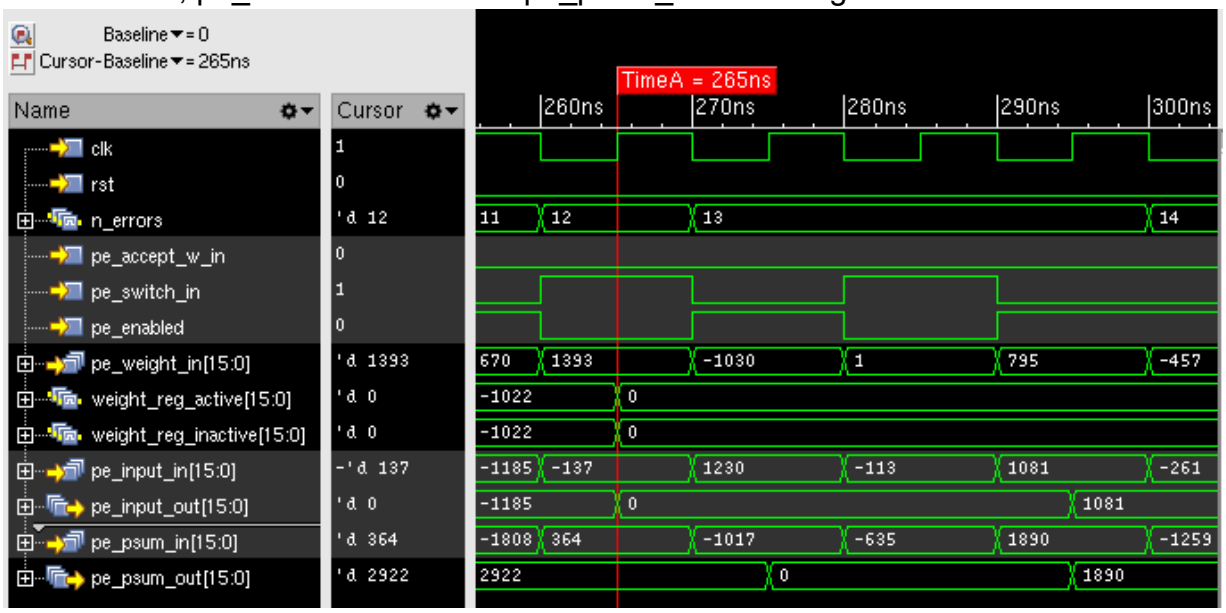


Spec this violates: “two copies of the weight: one it is using right now, and one waiting in the background until it is told to swap”
Fix: the weight should be promoted only when pe_switch_in is high. Currently, the vendor PE promotes weights immediately, regardless of the level of pe_switch_in.

pe_psum_out does not go to 0 when pe_enabled is low

```
MISMATCH #24: w=  1393 a=  -137 p=   364 | acc=0 sw=1 vld=0 en=0 expected=0 observed=2922
```

At the cursor, pe_enabled is low and pe_psum_out should go to 0.



Spec this violates: Alan: “When pe_enabled is 0 all five outputs should go to 0”
Fix: Drive pe_psum_out to 0 when pe_enabled is low

Random vs directed tests
Random stimulus found all 3 of these defects.

Half B Answers

1. The second half of the handshake (item_done) lets the sequence know when the driver has finished processing the transaction. The sequence waits for this handshake with finish_item(), so the generator doesn’t run ahead of the driver and only sends transactions when the driver has finished processing the previous transaction. In Half A, the generator queues a transaction into the mailbox, but receives no acknowledgement that the driver has finished processing it, so the generator continues running, regardless of if the driver has finished processing.
2. +define is a compile-time switch, so changing configurations requires you to recompile/have separate builds. In contrast, a factory override can select configurations/replacement classes at run time, which saves on overhead and doesn’t require editing the shared environment.
3. Using uvm_config_db allows you to pass the interface handle without having to add extra arguments/wiring to intermediate components to forward the interface handle across several hierarchy levels.